ISW - Tutorato

s.dessi63@studenti.unica.it

l.piras78@studenti.unica.it

OGGETTO MAIL [ISW]

Caratteristiche Python

- Python è considerato un linguaggio interpretato, anche se il codice sorgente non viene convertito direttamente in linguaggio macchina ma viene prima pre-compilato in bytecode (.pyc).
- E' orientato alla programmazione ad oggetti nel senso che ogni cosa su cui Python opera è un oggetto: tutti gli oggetti ereditano dall'oggetto base "object".
- E' un linguaggio che supporta diversi paradigmi/stili di programmazione: procedurale, funzionale, a oggetti, meta-programmazione, scripting.
- Implementa strutture complesse quali liste, dizionari, tuple, stringhe e numerose funzioni per agire su di esse.
- Contiene una grande libreria di funzioni, ed esistono in libera distribuzione tantissimi moduli Python.

Caratteristiche Python

- Python è case-sensitive
- I nomi delle variabili iniziano con un carattere alfabetico e contengono caratteri, numeri o underscore ‘_’
- I nomi che iniziano con ‘_’ hanno significati speciali
- Non è necessario il “;” dopo ogni linea di codice. Più istruzioni possono stare sulla stessa riga, se separate da punto e virgola ";"
- Quando una linea termina con il carattere: "\" allora continua nella successiva
- I commenti sono identificati dal carattere: "#“

Caratteristiche Python

- L’indentazione conta! caratteri bianchi all'inizio di una riga (diversi livelli di indentazione) sono utilizzati per definire blocchi logici
- indentare in modo incorretto può portare a comportamenti sbagliati del programma o a errori
- Esempio:
  - print("eseguito sempre all'inizio")
  - if condizione:
    - print('eseguito in mezzo solo se la condizione è vera’)
  - print('eseguito sempre alla fine’)

Caratteristiche

- Python ha un sistema di "garbage collection", per liberare memoria inutilizzata.
- binding dinamico: il controllo di esistenza di un metodo/attributo è fatto a run-time. Qesto meccanismo è diretta conseguenza del polimorfismo e dell’overriding.

Cosa vedremo oggi

- Istruzioni condizionali
- Cicli
- Funzioni
- Ricorsione
- Esercizi, esercizi, esercizi !

Reminder

Istruzioni condizionali: if elif else

- Le istruzioni condizionali vengono utilizzate quando vogliamo eseguire un blocco di codice solo nel caso in cui una condizione sia vera o falsa

```
if condizione :
        Istruzioni eseguite se la condizione è vera
elif condizione2:
        Istruzioni eseguite se la condizione2 è vera
else:
        Istruzioni eseguite se tutte le condizioni sono false
```

Istruzioni condizionali: if elif else

- Esempio:

```
n = int(input('Inserisci un numero:'))
if n < 0:
    print(n, 'è negativo')
elif n > 0:
    print(n, 'è positivo')
else:
    print(n, 'è zero')
```

Ciclo for

- Ci permette di iterare su tutti gli elementi di un iterabile (liste, tuple, set, dizionari, ecc.) ed eseguire un determinato blocco di codice

```
for variabile in iterabile :
        istruzioni
```

- Termina quando le istruzioni sono state eseguite per tutti i valori di variabile (quindi per tutti gli elementi di iterabile)
- Esempio:

```
seq = [1, 2, 3, 4, 5]
for n in seq:
        print('Il quadrato di', n, 'è', n**2)
```

Ciclo while

- Il ciclo while itera fintanto che la condizione è vera

```
while condizione :
    istruzioni
```

- Esempio:

```
# rimuovi e stampa numeri da seq finché ne rimangono solo 3
seq = [10, 20, 30, 40, 50, 60]
while len(seq) > 3:
    print(seq.pop())
```

Cicli: break e continue

- All'interno dei cicli possiamo utilizzare:
  - Break : interrompe il ciclo
  - Continue : interrompe l'iterazione corrente ed esegue quella successive

```
seq = ['alpha', 'beta', 'gamma', 'delta']
for elem in seq:
    if len(elem) == 4:
        continue  # procedi all'elemento successivo
    print('Sto controllando', elem)
    if elem == 'gamma':
        print('Elemento trovato!')
        break  # elemento trovato, interrompi il ciclo
```

Definizione di funzioni

- Una funzione e' in genere una sequenza di comandi, a cui viene dato un nome, che esegue una certa operazione
- Per definire una funzione:

```
def nome_funzione(param1, param2=10) :
    istruzioni
    [return]
```

Definizione di funzioni

- Una volta definita, è possibile chiamarla, semplicemente invocando il suo nome (nome_funzione) e passando i parametri necessari
- Esempio:

```
# definizione
def newLine():
    print() # o anche solo print
# chiamata
newLine()
```

Definizione di funzioni

- Esempio 2:

```
def somma(a=2, b=3):
        return a+b

s = somma(3,4)
print(s)
```

Input da tastiera

- Per prendere input da tastiera da parte dell'utente, usare le funzioni di libreria:

  - raw_input() # deprecato da Python 3
  - input() # prima per i numeri interi, ora per tutto

Input da tastiera

- Esempi:
  - prendere in input una parola:

```
parola = input('Inserire una parola: ')
```

  - prendere in input un numero:

```
num = (input('Inserire un numero: '))
```

Output su terminale

- Per stampare in output su terminale il risultato delle vostre elaborazioni, usare la funzione di libreria:
  - print()

- Esempio:

```
num = (input('Inserire un numero: ‘))
print(‘il numero inserito e\’: ‘, num)
print( ‘num = {} '.format(num))
```

Built-in functions

- Sono funzioni definite all'interno dell'interprete python.
  - int(): prende come argomento un valore numerico e restituisce un numero intero
  - float(): prende come argomento un valore numerico e restituisce un numero decimale
  - type(): prende come argomento un oggetto e ne restituisce il tipo
  - abs(): restituisce il valore assoluto di un valore numerico
  - dict(): restituisce un dizionario

Built-in functions

- getattr(): permette di chiamare un attributo o un metodo dell'istanza di una classe
- list(): converte un iterabile in una lista
- isinstance(): verifica se l'oggetto è realmente un'istanza della classe passata
- len(): restituisce la lunghezza dell'argomento passato
- enumerate(iterable, start=0): prende un iterabile e restituisce un oggetto numerato (0, elem0), (1, elem1), …

Built-in functions

- format(): restituisce un valore all'interno di una stringa
- range(start, stop, step=1): costruisce un intervallo numerico
- map(oggetto, iterabile): funzione di cast per iterabili
- print(): stampa su standard output
- min(): restituisce il valore minimo di un iterabile
- max(): restituisce il valore massimo di un iterabile

Built-in functions

- round(variabile, int): arrotonda il float inserito al numero di cifre decimali specificato
- sorted(iterabile): restituisce l'iterabile ordinato
- str(): converte la variabile passata in una stringa
- sum(iterabile): restituisce la somma degli elementi dell'iterabile passato
- tuple(): converte un iterabile in una tupla

Built-in functions

- zip(): prende due iterabili e li aggrega in un iterabile di tuple
- set(): restituisce un Python set
- reversed(): restituisce l'iterabile passatogli al contrario
- all(): prende un iterabile e restituisce True se tutti i suoi elementi sono True
- any(): restituisce True se almeno un elemento dell'iterabile è True